

Symbolic Artificial Intelligence (COMP3008)

Decision problems, formal languages and solvers

24 March 2026

Daniel Karapetyan

`daniel.karapetyan@nottingham.ac.uk`

Long-standing split in AI

Symbolic AI

Dominant until 90-s; gaining interest again

The model is **explicit** (e.g., in the form of rules)

The model is provided by a human expert

Generally interpretable

Formal reasoning

Non-symbolic AI

Dominant since 2000-s

The model is **implicit** (think of a neural network)

The model is automatically extracted from the patterns observed in the data

Generally uninterpretable

Statistical or implicit reasoning

Why is it symbolic?

Why *symbolic*?

The knowledge and rules are specified using symbols. Symbols can represent concepts, relationships, and logical formulas.

How to solve symbolic AI problems

We need:

- A formal language to describe such problems
- A human expert to formulate a problem in such a formal language
- A software package that can understand the language and solve the problem

Decision problems

Symbolic AI usually deals with *decision problems*

Decision problem:

- Described symbolically (i.e., via a mathematical model)
- The answer is 'yes' (solution exists) or 'no' (solution does not exist)
- Seeking any acceptable solution
 - As opposed to optimisation: seeking the best possible solution

Decision problems

Symbolic AI usually deals with *decision problems*

Decision problem:

- Described symbolically (i.e., via a mathematical model)
- The answer is 'yes' (solution exists) or 'no' (solution does not exist)
- Seeking any acceptable solution
 - As opposed to optimisation: seeking the best possible solution

Examples of decision problems:

- Given integer x , is x a prime number?
- Given a set of integers, is there a subset of it that adds up to 31?
- Given all the necessary data, can we schedule exams so that there are no timetable clashes?

The choice of the language

Trade-off:

Expressivity: how rich is the language (how many problems it can describe)

Performace: how efficient are the reasoning algorithms

First-order logic: high expressivity

First-order logic is a commonly used formal language to describe decision problems

Example: is the following expression satisfiable?

$$\forall x. (A(x) \rightarrow \exists y. \neg B(x, y))$$

First-order logic is *undecidable*: there is no algorithm that is guaranteed to arrive to an answer to every such question in finite time

Propositional Logic: better performance

First-order logic is too powerful to allow efficient reasoning

We often restrict it to the *propositional logic*

Example: is the following expression satisfiable:

$$(A \wedge \neg B) \rightarrow (\neg A \vee C)$$

Propositional logic is decidable but **computationally hard**
(NP-complete)

How bad is undecidability or computational hardness?

Observations:

- Even if a logic is undecidable or computationally hard in general, reasoning may often be manageable in practice
 - Intelligent reasoning algorithm may be able to prioritise the most important choices, etc.
- In some applications, such as automated theorem proving, we may be OK to wait for a very long time, even without guarantee of success, but we need high expressivity (usually full first-order logic)
- In other applications, such as exam timetabling, we may not need the expressivity of the first-order logic but we care about the solution time

SAT Solvers

If the model is described in the propositional logic, it is common to express the problem as SAT:

SAT is a decision problem where, given a propositional formula in conjunctive normal form (CNF), we seek an assignment of propositional variables that satisfies the formula or prove that the formula is unsatisfiable

SAT Solvers

If the model is described in the propositional logic, it is common to express the problem as SAT:

SAT is a decision problem where, given a propositional formula in conjunctive normal form (CNF), we seek an assignment of propositional variables that satisfies the formula or prove that the formula is unsatisfiable

Example of SAT:

Is this satisfiable? $(A \vee \neg B) \wedge (\neg A \vee C) \wedge (D)$

- The SAT problem is NP-complete (computationally hard)
- In practice, SAT solvers are extremely efficient
- Many general-purpose solvers actually convert other problems into SAT to exploit this efficiency

DPLL algorithm – background

Most of the modern SAT solvers are based on the DPLL algorithm, named after its creators Davis, Putnam, Logemann and Loveland

- Let S be our expression in CNF
- Observe that S involves a finite number of propositional variables
- We can potentially try every combination of their values
 - If we find a combination that satisfies S then we solved the problem
 - If we did not find any combination that satisfies S then S is unsatisfiable
- In practice, we might not need to enumerate all possible combinations
 - Once we have a partial assignment of values, we may already be able to conclude that this partial assignment does not satisfy S
 - For example, in $(A \vee \neg B) \wedge (\neg A \vee C) \wedge (D)$ setting $D = \text{FALSE}$ makes the formula unsatisfiable; then there is no need to try all combinations of A , B and C with $D = \text{FALSE}$

DPLL algorithm

- DPLL uses *Depth-First Search*
- It selects a single propositional variable and then splits the search into two branches: when this variable is FALSE and when it is TRUE
- It substitutes the value of the variable and simplifies the formula
- If any clause is FALSE, the formula is unsatisfiable and no further search in this branch is needed
- A few simple heuristics dramatically improve the performance of DPLL

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

■ Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

■ Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

■ Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{FALSE}) \wedge (\text{TRUE})$ – this branch is unsatisfiable.

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{FALSE}) \wedge (\text{TRUE})$ – this branch is unsatisfiable.

- Let $B = \text{TRUE}$:

$$(C) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}).$$

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{FALSE}) \wedge (\text{TRUE})$ – this branch is unsatisfiable.

- Let $B = \text{TRUE}$:

$$(C) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}).$$

We can set $C = \text{TRUE}$ (heuristic).

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{FALSE}) \wedge (\text{TRUE})$ – this branch is unsatisfiable.

- Let $B = \text{TRUE}$:

$$(C) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}).$$

We can set $C = \text{TRUE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE})$ – **satisfiable**.

DPLL algorithm – example

$$(A \vee \neg B \vee C) \wedge (A \vee B \vee \neg C) \wedge (B \vee C) \wedge (\neg A \vee B \vee C)$$

Select A :

- Let $A = \text{FALSE}$:

$$(\neg B \vee C) \wedge (B \vee \neg C) \wedge (B \vee C) \wedge (\text{TRUE}).$$

Select B :

- Let $B = \text{FALSE}$:

$$(\text{TRUE}) \wedge (\neg C) \wedge (C) \wedge (\text{TRUE}).$$

We can set $C = \text{FALSE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{FALSE}) \wedge (\text{TRUE})$ – this branch is unsatisfiable.

- Let $B = \text{TRUE}$:

$$(C) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}).$$

We can set $C = \text{TRUE}$ (heuristic).

Then $(\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE}) \wedge (\text{TRUE})$ – **satisfiable**.

- We do not need to try $A = \text{TRUE}$ as we have already proven that the problem is satisfiable.

Summary

- Symbolic AI solves decision problems
- A problem needs to be described in a formal language such as first-order logic
- First-order logic is undecidable in general but some subsets such as propositional logic are decidable
- SAT is a standard form for problems formulated using the propositional logic
- Most of the SAT solvers are based on DPLL

What you will learn in the Symbolic AI module

- How to formulate real-world problems using formal languages such as propositional logic
- How to use solvers (Microsoft Z3; Google OR-Tools)
- How those solvers work